

COMPONENTE CURRICULAR:	Desenvolvimento de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. Modos de Operação: Padrão State

Para a transição entre os modos "Economia", "Conforto" e "Ausente", o padrão **State** é o mais adequado.

- **Justificativa:** O comportamento das luzes e dos sensores de presença depende diretamente do **estado atual** do sistema. No State, o objeto altera seu comportamento conforme seu estado interno muda, parecendo que a classe do objeto mudou.
- **Aplicação no Cenário:** Cada modo de operação seria uma classe concreta que implementa uma interface comum. A mudança de um modo para outro (ex: de "Conforto" para "Ausente") altera como o sistema reage aos estímulos dos sensores, encapsulando essa lógica e evitando estruturas condicionais (if/else ou switch) gigantescas na classe principal.

2. Algoritmos de Consumo Energético: Padrão Strategy

Para a variação entre os algoritmos de cálculo (simples, detalhado ou preditivo), o padrão **Strategy** é a escolha correta.

- **Justificativa:** O foco aqui é a **execução de uma tarefa específica** (calcular consumo) de diferentes maneiras. O Strategy permite que você selecione um algoritmo em tempo de execução e o troque sem alterar o contexto que o utiliza.
- **Aplicação no Cenário:** Diferente do State, onde o estado dita o comportamento global, aqui temos uma ação pontual. O usuário ou o sistema decide *como* quer realizar o cálculo. Definimos uma interface de cálculo e cada algoritmo (Simples, Detalhado, Preditivo) é uma estratégia intercambiável. Isso segue o princípio de "composição sobre herança", facilitando a adição de novos métodos de cálculo no futuro.

Conclusão: Enquanto o **State** foca em *o que o sistema é* naquele momento, o **Strategy** foca em *como o sistema faz* uma operação específica. Ambos promovem o baixo acoplamento e facilitam a manutenção do código.